

# Capítulo 5: Redes neuronales

Parte 1: funciones feed-forward y entrenamiento

Basado en C. M. Bishop, *Pattern Recognition and Machine Learning*

- 1 Contexto
- 2 Funciones feed-forward
- 3 Expresividad y aproximación
- 4 Simetrías en el espacio de pesos
- 5 Entrenamiento
- 6 Cierre

## Forma general

$$y(\mathbf{x}, \mathbf{w}) = f\left(\sum_{j=1}^M w_j \phi_j(\mathbf{x})\right)$$

- ▶  $\mathbf{x} = (x_1, \dots, x_D)^\top \in \mathbb{R}^D$ : vector de entrada.
- ▶  $\phi_j(\mathbf{x})$ :  $j$ -ésima función base **fija**.
- ▶  $w_j \in \mathbb{R}$ : coeficiente asociado a la base  $\phi_j$ .
- ▶  $M$ : número de funciones base.
- ▶  $f(\cdot)$ : función de salida.
  - En regresión estándar,  $f$  puede ser la identidad.
  - En clasificación,  $f$  suele ser no lineal (por ejemplo sigmoide o softmax).

## ¿Por qué aparece la *maldición de la dimensionalidad*?

- ▶ Con bases fijas solemos cubrir el espacio de entrada con muchas regiones para captar estructura local.
- ▶ En dimensión alta, esa cobertura crece muy rápido.
- ▶ Si usamos una resolución parecida en cada coordenada, el número de celdas o regiones escala como

$$K^D,$$

donde  $K$  es el número de divisiones por coordenada y  $D$  la dimensión de entrada.

- ▶ Por tanto, una colección grande de bases fijas se vuelve costosa en datos, memoria y cómputo.

### Idea clave

Necesitamos **adaptar la representación al problema** en vez de cubrir uniformemente todo el espacio de entrada.

## Bases fijas

$$\phi_j(\mathbf{x}) = \exp\left(-\frac{\|\mathbf{x} - \boldsymbol{\mu}_j\|^2}{2s^2}\right)$$

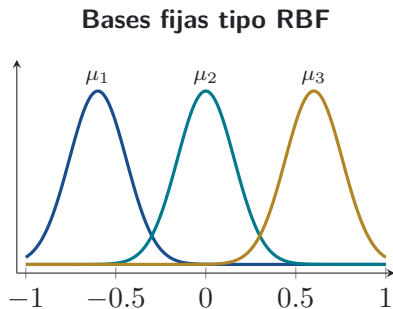
- ▶ Los centros  $\boldsymbol{\mu}_j$  se fijan de antemano.
- ▶ La forma de la base no aprende de los datos.
- ▶ Una estructura compleja puede requerir muchas bases.

## Bases adaptativas (paramétricas)

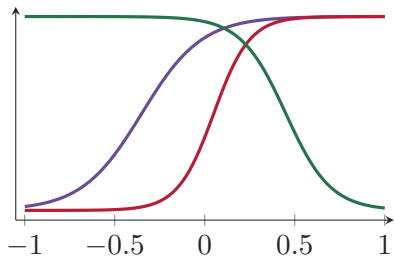
$$\phi_j(\mathbf{x}; \theta_j) = h(\mathbf{a}_j^\top \mathbf{x} + b_j)$$

- ▶  $\theta_j = (\mathbf{a}_j, b_j)$  son parámetros ajustables.
- ▶ La base cambia orientación y posición durante el entrenamiento.
- ▶ La representación se concentra en regiones relevantes.

## Figura: bases fijas vs. bases adaptativas



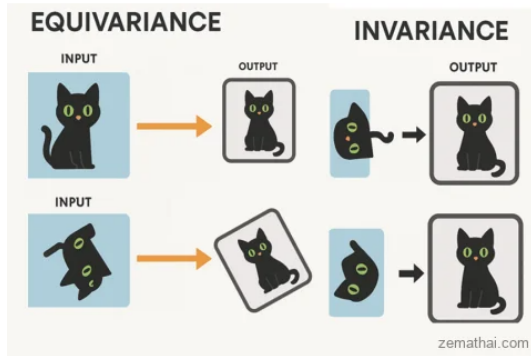
**Bases adaptativas tipo  $h(a^\top x + b)$**



Las bases fijas cubren de antemano el espacio; las adaptativas se **reposicionan y reorientan** para capturar mejor la estructura relevante. RBF: Radial basis function.

## Idea clave

La red aprende representaciones intermedias útiles para la tarea de salida mediante sus parámetros internos, en particular los pesos y sesgos de las capas ocultas.



**Figura 1:** Ejemplo de representaciones intermedias útiles en un problema de clasificación de imágenes faciales

- ▶ En regresión, las unidades ocultas pueden actuar como bloques básicos cuya combinación aproxima una función objetivo.
- ▶ En clasificación, las unidades ocultas pueden definir regiones, umbrales suaves o detectores de patrones en el espacio de entrada.
- ▶ La representación intermedia no tiene por qué ser “interpretable” semánticamente; su valor está en que hace más sencilla la tarea final para la capa de salida.

### Definición

$$\mathbf{x} \longmapsto \mathbf{z}(\mathbf{x}) \longmapsto \mathbf{y}(\mathbf{x})$$

Aquí  $\mathbf{z}(\mathbf{x}) = (z_1(\mathbf{x}), \dots, z_M(\mathbf{x}))^\top$  es la representación intermedia aprendida.

- ▶ Partimos del modelo lineal con bases:

$$y(\mathbf{x}, \mathbf{w}) = f\left(\sum_{j=1}^M w_j \phi_j(\mathbf{x})\right).$$

- ▶ Si ahora cada base es ella misma una función paramétrica de la entrada,

$$\phi_j(\mathbf{x}) = h\left(\sum_{i=1}^D w_{ji}^{(1)} x_i + w_{j0}^{(1)}\right),$$

entonces al sustituir obtenemos una función **anidada**.

- ▶ Esa anidación produce naturalmente una **composición de transformaciones lineales y no lineales**.

## Definición

$$a_j = \sum_{i=1}^D w_{ji}^{(1)} x_i + w_{j0}^{(1)}, \quad j = 1, \dots, M,$$

$$z_j = h(a_j).$$

- ▶  $D$ : dimensión de la entrada.
- ▶  $M$ : número de unidades ocultas.
- ▶  $w_{ji}^{(1)}$ : peso desde la entrada  $x_i$  hacia la unidad oculta  $j$ .
- ▶  $w_{j0}^{(1)}$ : sesgo de la unidad oculta  $j$ .
- ▶  $a_j$ : activación antes de la no linealidad.
- ▶  $h(\cdot)$ : función de activación oculta; típicamente sigmoide o tanh.

### Definición

$$a_k = \sum_{j=1}^M w_{kj}^{(2)} z_j + w_{k0}^{(2)}, \quad k = 1, \dots, K,$$
$$y_k = g(a_k).$$

- ▶  $K$ : número de salidas.
- ▶  $w_{kj}^{(2)}$ : peso desde la unidad oculta  $j$  hacia la salida  $k$ .
- ▶  $w_{k0}^{(2)}$ : sesgo de la salida  $k$ .
- ▶  $g(\cdot)$ : activación de salida; depende del tipo de problema.

$$y_k(\mathbf{x}, \mathbf{w}) = g \left( \sum_{j=1}^M w_{kj}^{(2)} h \left( \sum_{i=1}^D w_{ji}^{(1)} x_i + w_{j0}^{(1)} \right) + w_{k0}^{(2)} \right)$$

- ▶  $\mathbf{w}$  denota el vector formado por *todos* los pesos y sesgos de la red.
- ▶ La red es una función determinista

$$\mathbf{x} \mapsto \mathbf{y}(\mathbf{x}, \mathbf{w})$$

parametrizada por  $\mathbf{w}$ .

## Regresión

$$g(a) = a$$

$$y_k = a_k$$

## Binaria

$$g(a) = \sigma(a) = \frac{1}{1 + e^{-a}}$$

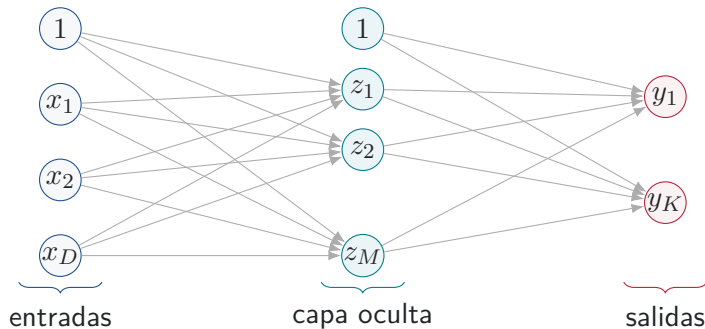
$$y_k = \sigma(a_k)$$

## Multiclase

$$g(\mathbf{a}) = \text{softmax}(\mathbf{a})$$

$$y_k = \frac{e^{a_k}}{\sum_j e^{a_j}}$$

# Diagrama de una red de dos capas



La evaluación de la salida corresponde a una **propagación hacia adelante** de la información.

- ▶ Definimos una entrada adicional fija  $x_0 = 1$ .
- ▶ Entonces la activación oculta puede escribirse como

$$a_j = \sum_{i=0}^D w_{ji}^{(1)} x_i.$$

- ▶ De manera análoga, definimos  $z_0 = 1$  para absorber los sesgos de la capa de salida:

$$a_k = \sum_{j=0}^M w_{kj}^{(2)} z_j.$$

## Idea

La notación se simplifica y las derivaciones posteriores de gradientes quedan más limpias.

- ▶ La arquitectura más común tiene:
  - una capa de entrada,
  - una capa oculta,
  - una capa de salida.
- ▶ Sin embargo, Bishop recomienda contar las **capas de pesos adaptativos**.
- ▶ Por ello, esta arquitectura se denomina **red de dos capas**.

## Observación

También pueden considerarse más capas ocultas, conexiones *skip-layer* y topologías feed-forward más generales.

## Definición informal

La expresividad de una arquitectura es la riqueza de funciones que puede representar o aproximar con alguna elección de parámetros.

- ▶ Una clase de modelos poco expresiva solo puede describir un conjunto restringido de funciones.
- ▶ Una clase más expresiva puede capturar curvaturas, fronteras de decisión complejas o dependencias no lineales.
- ▶ En una red neuronal, la expresividad depende de:
  - la activación no lineal,
  - el número de unidades ocultas,
  - el número de capas,
  - la arquitectura de conexiones.

Si  $h(a) = a$  para todas las unidades ocultas, entonces

$$\mathbf{z} = W^{(1)}\mathbf{x} + \mathbf{b}^{(1)}, \quad \mathbf{y} = W^{(2)}\mathbf{z} + \mathbf{b}^{(2)}.$$

Sustituyendo,

$$\mathbf{y} = W^{(2)}W^{(1)}\mathbf{x} + W^{(2)}\mathbf{b}^{(1)} + \mathbf{b}^{(2)} = \widetilde{W}\mathbf{x} + \widetilde{\mathbf{b}}.$$

### Advertencia

La composición de transformaciones lineales sigue siendo lineal.

La no linealidad en las capas ocultas es indispensable para ganar expresividad no lineal.

## Idea informal

Una red feed-forward con una sola capa oculta y suficientes unidades puede aproximar arbitrariamente bien funciones continuas sobre conjuntos compactos, bajo condiciones adecuadas sobre la activación.

- ▶ Es un resultado de **existencia**.
- ▶ No dice cuántas unidades ocultas harán falta en un problema dado.
- ▶ No garantiza que el entrenamiento sea fácil.
- ▶ No sustituye el análisis de generalización.

- ▶ La idea matemática es mostrar que la familia generada por combinaciones finitas de la forma

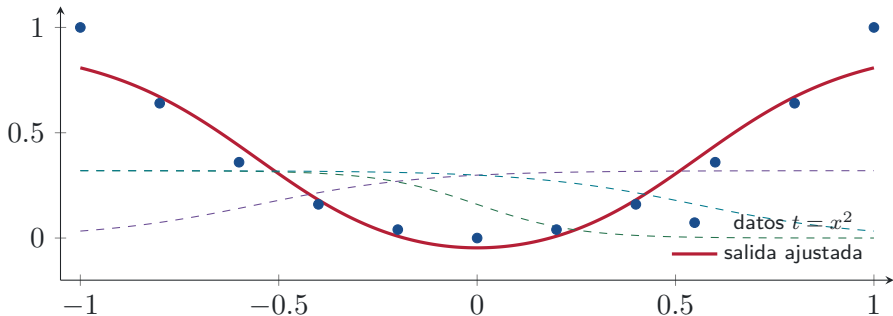
$$\sum_{j=1}^M \alpha_j h(\mathbf{a}_j^\top \mathbf{x} + b_j)$$

es densa en una clase de funciones objetivo.

- ▶ “Densa” significa que, dada una función objetivo  $f$  y un error tolerable  $\varepsilon > 0$ , existe una red cuya salida queda a distancia menor que  $\varepsilon$  de  $f$  en el sentido considerado.
- ▶ El resultado depende del tipo de activación y del espacio funcional donde se mida el error.

- ▶ G. Cybenko, "Approximation by superpositions of a sigmoidal function," *Mathematics of Control, Signals, and Systems*, 2(4), 303–314, 1989.
- ▶ K.-I. Funahashi, "On the approximate realization of continuous mappings by neural networks," *Neural Networks*, 2(3), 183–192, 1989.
- ▶ K. Hornik, M. Stinchcombe y H. White, "Multilayer feedforward networks are universal approximators," *Neural Networks*, 2(5), 359–366, 1989.
- ▶ K. Hornik, "Approximation capabilities of multilayer feedforward networks," *Neural Networks*, 4(2), 251–257, 1991.

## Figura: activaciones ocultas y salida de la red



Las curvas discontinuas ilustran *activaciones ocultas reescaladas para visualización*; la curva roja muestra una combinación lineal de unidades ocultas que aproxima la función objetivo no lineal.

- ▶ Cuando digamos que el MLP (Multi Layer Perceptron o Red Neuronal de varias capas) es más expresivo que un modelo lineal, queremos decir que puede representar relaciones entrada-salida mucho más complejas.
- ▶ En 1D: puede aproximar funciones no lineales con distintos tipos de curvatura.
- ▶ En clasificación 2D: puede inducir fronteras de decisión no lineales.
- ▶ En términos de representación, la capa oculta transforma el espacio original en otro donde la tarea final puede volverse más simple.

### Idea

Mayor expresividad no implica automáticamente mejor rendimiento fuera de muestra; también aumenta el riesgo de sobreajuste y la dificultad de optimización.

Supongamos activaciones ocultas  $\tanh$ , de modo que

$$\tanh(-a) = -\tanh(a).$$

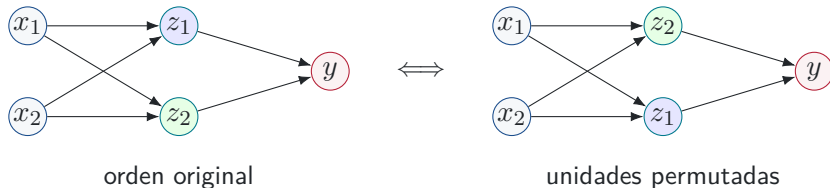
Si cambiamos el signo de todos los pesos y el sesgo que llegan a una unidad oculta  $j$ , entonces su salida cambia de signo.

Si simultáneamente cambiamos el signo de todos los pesos que salen de esa unidad, la función global de la red permanece igual.

### Advertencia

Distintos vectores de pesos pueden representar exactamente la misma función entrada - salida.

## Simetría por permutación de unidades ocultas



- ▶ Intercambiar dos unidades ocultas y permutar coherentemente sus pesos entrantes y salientes no cambia la función.
- ▶ Con  $M$  unidades ocultas hay  $M!$  permutaciones posibles.
- ▶ Con la simetría de signo, el factor total clásico es  $M!2^M$ .

- ▶ El paisaje de optimización contiene muchas descripciones paramétricas equivalentes.
- ▶ Dos soluciones numéricas diferentes pueden implementar exactamente la misma función.
- ▶ Esto es relevante para:
  - la interpretación geométrica del error,
  - la comparación entre mínimos locales,
  - la comparación bayesiana de modelos.

- ▶ Tenemos  $N$  pares de entrenamiento

$$\mathcal{D} = \{(\mathbf{x}_n, \mathbf{t}_n)\}_{n=1}^N.$$

- ▶  $\mathbf{x}_n$ : entrada del patrón  $n$ .
- ▶  $\mathbf{t}_n$ : blanco o *target* asociado.
- ▶ La salida de la red para el patrón  $n$  es

$$\mathbf{y}(\mathbf{x}_n, \mathbf{w}).$$

## Idea

Elegimos una distribución condicional  $p(\mathbf{t} \mid \mathbf{x}, \mathbf{w})$  y entrenamos por máxima verosimilitud.

Supongamos primero un target escalar  $t \in \mathbb{R}$ .

## Modelo

$$p(t \mid \mathbf{x}, \mathbf{w}, \beta) = \mathcal{N}(t \mid y(\mathbf{x}, \mathbf{w}), \beta^{-1})$$

- ▶  $y(\mathbf{x}, \mathbf{w})$ : media condicional predicha por la red.
- ▶  $\beta > 0$ : precisión del ruido gaussiano ( $\beta^{-1}$  es la varianza).
- ▶ En este caso, la activación de salida natural es la **identidad**.

Para datos i.i.d.:

$$p(\mathbf{t} \mid X, \mathbf{w}, \beta) = \prod_{n=1}^N p(t_n \mid \mathbf{x}_n, \mathbf{w}, \beta).$$

Tomando logaritmo negativo:

$$-\log p(\mathbf{t} \mid X, \mathbf{w}, \beta) = \frac{\beta}{2} \sum_{n=1}^N (y(\mathbf{x}_n, \mathbf{w}) - t_n)^2 - \frac{N}{2} \log \beta + \frac{N}{2} \log(2\pi).$$

Si queremos optimizar respecto a  $\mathbf{w}$  y tratamos  $\beta$  como constante, esto equivale a minimizar

$$E(\mathbf{w}) = \frac{1}{2} \sum_{n=1}^N (y(\mathbf{x}_n, \mathbf{w}) - t_n)^2.$$

- ▶ La suma de cuadrados no se elige por defecto.
- ▶ Surge como **negativo del logaritmo de una verosimilitud**.

Ahora  $t \in \{0, 1\}$ , donde por ejemplo:

$$t = 1 \iff C_1, \quad t = 0 \iff C_2.$$

## Modelo

$$y(\mathbf{x}, \mathbf{w}) = \sigma(a) = \frac{1}{1 + e^{-a}},$$

$$p(t \mid \mathbf{x}, \mathbf{w}) = y(\mathbf{x}, \mathbf{w})^t [1 - y(\mathbf{x}, \mathbf{w})]^{1-t}.$$

Interpretamos  $y(\mathbf{x}, \mathbf{w})$  como la probabilidad condicional de la clase positiva.

Para datos i.i.d.:

$$p(\mathbf{t} \mid X, \mathbf{w}) = \prod_{n=1}^N y_n^{t_n} (1 - y_n)^{1-t_n},$$

con  $y_n = y(\mathbf{x}_n, \mathbf{w})$ .

Tomando logaritmo negativo:

$$E(\mathbf{w}) = - \sum_{n=1}^N [t_n \log y_n + (1 - t_n) \log(1 - y_n)].$$

## Nota

Esta función de error se llama **entropía cruzada binaria** o **log-loss Bernoulli**.

Si la distribución verdadera sobre la etiqueta binaria es  $p$  y el modelo predice  $q$ , la entropía cruzada es

$$H(p, q) = - \sum_{t \in \{0,1\}} p(t) \log q(t).$$

Para un ejemplo con target  $t \in \{0, 1\}$ , se reduce a

$$- [t \log y + (1 - t) \log(1 - y)].$$

## Idea

Penaliza con fuerza predicciones muy confiadas y equivocadas.

- ▶ Minimizar entropía cruzada equivale a maximizar la verosimilitud Bernoulli.
- ▶ También equivale a acercar la distribución predicha a la distribución empírica de las etiquetas.
- ▶ Cuando el target es estrictamente 0 o 1, la pérdida vale casi 0 si el modelo asigna probabilidad cercana a 1 a la clase correcta.
- ▶ La pérdida crece mucho si el modelo asigna probabilidad muy pequeña a la clase correcta.

En el caso binario,

$$E = -[t \log y + (1 - t) \log(1 - y)], \quad y = \sigma(a).$$

Aplicando regla de la cadena:

$$\frac{\partial E}{\partial a} = \frac{\partial E}{\partial y} \frac{\partial y}{\partial a}.$$

Como

$$\frac{\partial E}{\partial y} = -\frac{t}{y} + \frac{1 - t}{1 - y}, \quad \frac{\partial y}{\partial a} = y(1 - y),$$

obtenemos

$$\frac{\partial E}{\partial a} = y - t.$$

Supongamos  $K$  etiquetas binarias independientes dadas  $\mathbf{x}$ :

$$\mathbf{t} = (t_1, \dots, t_K)^\top, \quad t_k \in \{0, 1\}.$$

El modelo es

$$p(\mathbf{t} \mid \mathbf{x}, \mathbf{w}) = \prod_{k=1}^K y_k(\mathbf{x}, \mathbf{w})^{t_k} [1 - y_k(\mathbf{x}, \mathbf{w})]^{1-t_k},$$

con  $y_k = \sigma(a_k)$ .

La pérdida total queda

$$E(\mathbf{w}) = - \sum_{n=1}^N \sum_{k=1}^K [t_{nk} \log y_{nk} + (1 - t_{nk}) \log(1 - y_{nk})].$$

## Ejemplo: etiquetado de imágenes

Dada una fotografía, queremos predecir simultáneamente si contiene:

persona, perro, árbol, automóvil, ...

- ▶  $\mathbf{x}_n$ : la imagen  $n$ .
- ▶  $t_{nk} = 1$ : la etiqueta  $k$  está presente en la imagen  $n$ .
- ▶  $t_{nk} = 0$ : la etiqueta  $k$  no está presente.
- ▶  $y_{nk}$ : probabilidad predicha de presencia de la etiqueta  $k$  en la imagen  $n$ .

## Diferencia con multiclase

Aquí varias etiquetas pueden valer 1 simultáneamente.

## ¿Por qué compartir la representación en el caso multietiqueta?

- ▶ En un modelo lineal independiente, cada etiqueta se resolvería por separado.
- ▶ En una red neuronal, la capa oculta puede aprender una representación compartida:

$$\mathbf{x} \mapsto \mathbf{z}(\mathbf{x}).$$

- ▶ Esto permite reutilizar rasgos útiles para varias etiquetas a la vez.
- ▶ Ejemplo: detectar una textura o una silueta puede ayudar simultáneamente a varias predicciones binarias.

Ahora cada entrada pertenece a exactamente una de  $K$  clases mutuamente excluyentes.

## Codificación 1-de- $K$

$$t_k \in \{0, 1\}, \quad \sum_{k=1}^K t_k = 1.$$

Usamos una salida softmax:

$$y_k(\mathbf{x}, \mathbf{w}) = \frac{\exp(a_k(\mathbf{x}, \mathbf{w}))}{\sum_{j=1}^K \exp(a_j(\mathbf{x}, \mathbf{w}))}.$$

Así,  $0 \leq y_k \leq 1$  y  $\sum_k y_k = 1$ .

Si interpretamos  $y_k(\mathbf{x}, \mathbf{w}) = p(t_k = 1 \mid \mathbf{x}, \mathbf{w})$ , la pérdida viene de la verosimilitud categórica:

$$E(\mathbf{w}) = - \sum_{n=1}^N \sum_{k=1}^K t_{nk} \log y_k(\mathbf{x}_n, \mathbf{w}).$$

- ▶ Como  $t_n$  tiene codificación 1-de- $K$ , solo sobrevive el término de la clase correcta.
- ▶ También aquí la pérdida es una **entropía cruzada**, ahora entre la distribución empírica categórica y la distribución softmax predicha.

Para un ejemplo con clase correcta  $c$ , la pérdida por patrón es

$$-\log y_c.$$

- ▶ Si la red asigna gran probabilidad a la clase correcta, la pérdida es pequeña.
- ▶ Si la red asigna probabilidad pequeña a la clase correcta, la pérdida es grande.
- ▶ La estructura softmax fuerza competencia entre clases, a diferencia del caso multietiqueta independiente.

Con softmax y entropía cruzada multiclase,

$$E = - \sum_{j=1}^K t_j \log y_j, \quad y_j = \frac{e^{a_j}}{\sum_{\ell} e^{a_{\ell}}}.$$

Usando la derivada del softmax,

$$\frac{\partial y_j}{\partial a_k} = y_j (\delta_{jk} - y_k),$$

se obtiene

$$\frac{\partial E}{\partial a_k} = y_k - t_k.$$

## Idea

La misma forma aparece en regresión, binaria y multiclase cuando se usa el emparejamiento natural entre activación de salida y verosimilitud.

Con salida lineal  $y_k = a_k$  y pérdida cuadrática

$$E = \frac{1}{2} \sum_{k=1}^K (y_k - t_k)^2,$$

para cada salida  $k$  tenemos

$$\frac{\partial E}{\partial a_k} = \frac{\partial E}{\partial y_k} \frac{\partial y_k}{\partial a_k} = (y_k - t_k) \cdot 1 = y_k - t_k.$$

| Tarea         | Activación de salida     | Modelo probabilístico                            | Función de error                    |
|---------------|--------------------------|--------------------------------------------------|-------------------------------------|
| Regresión     | identidad                | Gaussiana<br>$\mathcal{N}(t \mid y, \beta^{-1})$ | suma de cuadrados                   |
| Binaria       | sigmoide                 | Bernoulli                                        | entropía cruzada binaria            |
| Multietiqueta | sigmoides independientes | producto de Bernoulli                            | suma de entropías cruzadas binarias |
| Multiclase    | softmax                  | categórica 1-de- $K$                             | entropía cruzada multiclase         |

1. Los modelos con bases fijas sufren en alta dimensión; por eso resulta natural introducir bases adaptativas.
2. Una red feed-forward puede verse como una composición de transformaciones lineales y no lineales.
3. La capa oculta aprende una representación intermedia útil para la tarea final.
4. La no linealidad en las capas ocultas es la fuente esencial de expresividad no lineal.
5. La elección de activación de salida y función de error se justifica mediante máxima verosimilitud.
6. En tres casos importantes aparece la identidad

$$\frac{\partial E}{\partial a_k} = y_k - t_k,$$

que será el punto de partida de backpropagation.

## En resumen

Ya sabemos:

- ▶ qué función representa la red,
- ▶ qué significan sus parámetros,
- ▶ y qué función de error corresponde a cada tarea.

Ahora falta responder:

¿cómo calcular gradientes de manera eficiente para entrenar la red?

Próxima sesión: optimización, regla de la cadena y error backpropagation.

## Apéndice: Teorema de aproximación universal de Cybenko (1989)

Sea  $I_n = [0, 1]^n$  y sea  $C(I_n)$  el espacio de funciones continuas en  $I_n$  con la norma supremo

$$\|f\|_\infty = \sup_{\mathbf{x} \in I_n} |f(\mathbf{x})|.$$

### Definición (función sigmoideal)

Una función continua  $\sigma : \mathbb{R} \rightarrow \mathbb{R}$  es sigmoideal si

$$\lim_{t \rightarrow -\infty} \sigma(t) = 0, \quad \lim_{t \rightarrow +\infty} \sigma(t) = 1.$$

### Teorema (versión clásica, informalmente formalizada)

Si  $\sigma$  es sigmoideal y discriminatoria, entonces el conjunto de sumas finitas

$$G = \left\{ \sum_{j=1}^M \alpha_j \sigma(\mathbf{w}_j^\top \mathbf{x} + \theta_j) : M \in \mathbb{N} \right\}$$

es denso en  $C(I_n)$ .

### Densidad en $C(I_n)$

Decir que  $G$  es denso en  $C(I_n)$  significa que, para toda función continua  $f \in C(I_n)$  y todo  $\varepsilon > 0$ , existe una red de la forma

$$g(\mathbf{x}) = \sum_{j=1}^M \alpha_j \sigma(\mathbf{w}_j^\top \mathbf{x} + \theta_j)$$

tal que

$$\|f - g\|_\infty < \varepsilon.$$

- ▶ La aproximación es uniforme sobre el compacto  $I_n$ .
- ▶ La profundidad sigue siendo una sola capa oculta.
- ▶ La complejidad se desplaza al número de unidades  $M$ .

1. Suponer que  $G$  no es denso en  $C(I_n)$ .
2. Por Hahn–Banach, existe un funcional lineal continuo no nulo que anula a todo elemento de  $G$ .
3. Por Riesz, ese funcional puede representarse mediante una medida firmada  $\mu$  sobre  $I_n$ .
4. Entonces se tendría
$$\int_{I_n} \sigma(\mathbf{w}^\top \mathbf{x} + \theta) d\mu(\mathbf{x}) = 0 \quad \forall \mathbf{w}, \theta.$$
5. La propiedad discriminatoria de  $\sigma$  fuerza entonces  $\mu = 0$ .
6. Esto contradice que el funcional fuese no nulo.

### Idea clave

La no linealidad sigmoideal genera una familia de “ridge functions” suficientemente rica para separar medidas no nulas.

- ▶ Funahashi (1989) prueba resultados análogos para aproximación de funciones continuas mediante redes de tres capas en compactos.
- ▶ Hornik, Stinchcombe y White (1989) muestran que la propiedad no depende de una activación sigmoideal específica; una familia amplia de activaciones no polinomiales sirve para aproximación universal.
- ▶ Hornik (1991) profundiza en capacidades de aproximación en distintos espacios funcionales y bajo condiciones más generales.

## Advertencia

El teorema justifica *capacidad de representación*, pero no resuelve por sí solo tres problemas prácticos: cuántas unidades usar, cómo entrenarlas y cómo generalizar bien.